



PYTHON PROGRAMMING

Assignment



Submitted By: Shirsendu Munshi

REG. NO.: 0912312030166

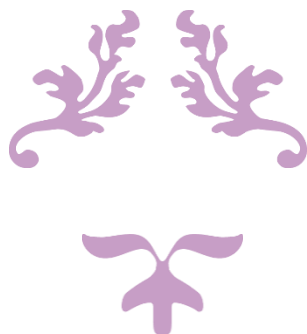
ROLL NO.: BCA/23/10

JANUARY 7, 2026
NORTH BENGAL ST. XAVIER'S COLLEGE

INDEX

PYTHON PROGRAMMING

Sl. No.	Title of the Program	Page No.	Teacher's Sign.
1	Menu Driven Program for Temperature Conversion (Fahrenheit & Celsius)	2	
2	Calculate Total Marks, Percentage, and Grade	2-3	
3	Menu Driven Program to Find Area of Different Shapes	3	
4	Display the first 'n' terms of the Fibonacci Series	4	
5	Find Factorial of a given number	4	
6	Implementation of Arrays using Python Lists	4	
7	Implementation of String Manipulation Techniques	5	
8	Calculate Sum of the Series: $1 - 2/2! + 3/3! + \dots + n/n!$	5	
9	Sum and Product of two compatible Matrices	6-7	
10	Implementation of Classes and Objects in Python	7	
11	Demonstration of Data Hiding and Private Variables	7	
12	Implementation of Class Constructor and Destructor	8	
13	Comprehensive Study of Different Types of Inheritance	8-9	
14	Concept of Method Overriding in Python	10	
15	Design of Mathematical 3D Objects using Classes (Sphere, Cone, etc.)	10-12	
16	Check if a Number is Positive, Negative, or Zero	12	
17	Check whether a given Year is a Leap Year	12	
18	Display the Calendar for a specific Month and Year	12	



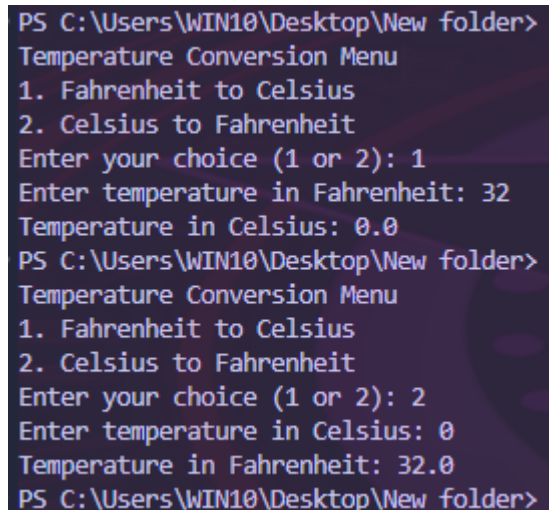
Python programs

1 WRITE A MENU DRIVEN PROGRAM TO CONVERT THE GIVEN TEMPERATURE FROM FAHRENHEIT TO CELSIUS AND VICE VERSA DEPENDING UPON USERS' CHOICE.

1.1 PROGRAM CODE:

```
# Menu Driven Program for Temperature Conversion
print("Temperature Conversion Menu")
print("1. Fahrenheit to Celsius")
print("2. Celsius to Fahrenheit")
choice = int(input("Enter your choice (1 or 2): "))
if choice == 1:
    f = float(input("Enter temperature in Fahrenheit: "))
    c = (f - 32) * 5 / 9
    print("Temperature in Celsius:", c)
elif choice == 2:
    c = float(input("Enter temperature in Celsius: "))
    f = (c * 9 / 5) + 32
    print("Temperature in Fahrenheit:", f)
else:
    print("Invalid choice! Please select 1 or 2.")
```

1.2 OUTPUT:



```
PS C:\Users\WIN10\Desktop\New folder>
Temperature Conversion Menu
1. Fahrenheit to Celsius
2. Celsius to Fahrenheit
Enter your choice (1 or 2): 1
Enter temperature in Fahrenheit: 32
Temperature in Celsius: 0.0
PS C:\Users\WIN10\Desktop\New folder>
Temperature Conversion Menu
1. Fahrenheit to Celsius
2. Celsius to Fahrenheit
Enter your choice (1 or 2): 2
Enter temperature in Celsius: 0
Temperature in Fahrenheit: 32.0
PS C:\Users\WIN10\Desktop\New folder>
```

2 WAP TO CALCULATE TOTAL MARKS, PERCENTAGE AND GRADE

2.1 PROGRAM CODE:

```
# Program to calculate total marks, percentage and grade
m1 = float(input("Enter marks of Subject 1: "))
m2 = float(input("Enter marks of Subject 2: "))
m3 = float(input("Enter marks of Subject 3: "))
total = m1 + m2 + m3
percentage = (total / 300) * 100
```

```

if percentage >= 80:
    grade = "A"
elif percentage >= 70:
    grade = "B"
elif percentage >= 60:
    grade = "C"
elif percentage >= 40:
    grade = "D"
else:
    grade = "E"
print("Total Marks:", total)
print("Percentage:", percentage)
print("Grade:", grade)

```

```

PS C:\Users\WIN10\Desktop\New folder>
Enter marks of Subject 1: 98
Enter marks of Subject 2: 95
Enter marks of Subject 3: 100
Total Marks: 293.0
Percentage: 97.66666666666667
Grade: A

```

3 MENU DRIVEN PROGRAM TO FIND AREA OF DIFFERENT SHAPES

3.1 PROGRAM CODE:

```

# User-defined functions
def area_rectangle(length, breadth):
    return length * breadth
def area_square(side):
    return side * side
def area_circle(radius):
    return 3.14 * radius * radius
def area_triangle(base, height):
    return 0.5 * base * height
# Menu Driven Program
print("Area Calculation Menu")
print("1. Area of Rectangle")
print("2. Area of Square")
print("3. Area of Circle")
print("4. Area of Triangle")
choice = int(input("Enter your choice (1-4): "))
if choice == 1:
    l = float(input("Enter length: "))
    b = float(input("Enter breadth: "))
    print("Area of Rectangle:", area_rectangle(l, b))
elif choice == 2:
    s = float(input("Enter side: "))
    print("Area of Square:", area_square(s))
elif choice == 3:
    r = float(input("Enter radius: "))
    print("Area of Circle:", area_circle(r))
elif choice == 4:
    base = float(input("Enter base: "))
    height = float(input("Enter height: "))
    print("Area of Triangle:", area_triangle(base, height))
else:
    print("Invalid choice!")

```

```

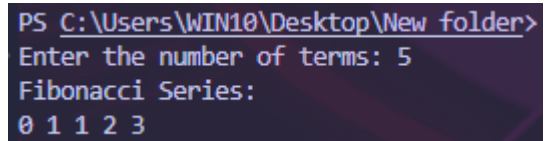
PS C:\Users\WIN10\Desktop\New folder>
Area Calculation Menu
1. Area of Rectangle
2. Area of Square
3. Area of Circle
4. Area of Triangle
Enter your choice (1-4): 1
Enter length: 12
Enter breadth: 4
Area of Rectangle: 48.0

```

4 WAP TO DISPLAY THE FIRST N TERMS OF FIBONACCI SERIES.

4.1 PROGRAM CODE:

```
# Program to print first n terms of Fibonacci series
n = int(input("Enter the number of terms: "))
a = 0
b = 1
print("Fibonacci Series:")
for i in range(n):
    print(a, end=" ")
    a, b = b, a + b
```

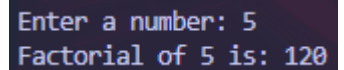


```
PS C:\Users\WIN10\Desktop\New folder>
Enter the number of terms: 5
Fibonacci Series:
0 1 1 2 3
```

5 WAP TO FIND FACTORIAL OF THE GIVEN NUMBER.

5.1 PROGRAM CODE:

```
# Program to find factorial of a number
n = int(input("Enter a number: "))
fact = 1
for i in range(1, n + 1):
    fact = fact * i
print("Factorial of", n, "is:", fact)
```

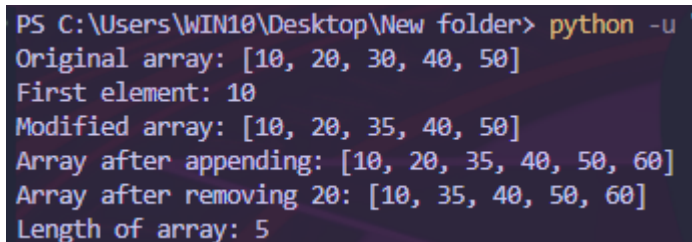


```
Enter a number: 5
Factorial of 5 is: 120
```

6 WAP TO IMPLEMENT THE USE OF ARRAYS IN PYTHON.

6.1 PROGRAM CODE:

```
# Using Lists (Dynamic Arrays)
# Creating and using a list (array)
numbers = [10, 20, 30, 40, 50]
print("Original array:", numbers)
# Access elements
print("First element:", numbers[0])
# Modify an element
numbers[2] = 35
print("Modified array:", numbers)
# Add a new element
numbers.append(60)
print("Array after appending:", numbers)
# Remove an element
numbers.remove(20)
print("Array after removing 20:", numbers)
# Length of array
print("Length of array:", len(numbers))
```

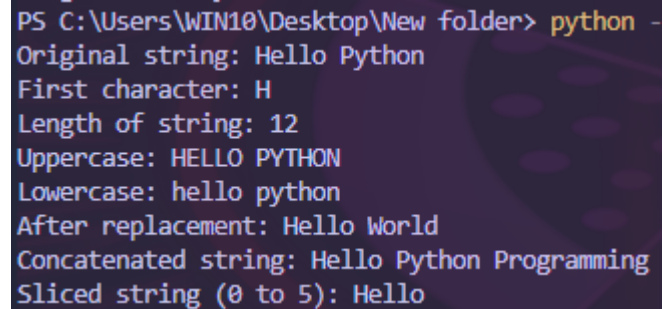


```
PS C:\Users\WIN10\Desktop\New folder> python -u
Original array: [10, 20, 30, 40, 50]
First element: 10
Modified array: [10, 20, 35, 40, 50]
Array after appending: [10, 20, 35, 40, 50, 60]
Array after removing 20: [10, 35, 40, 50, 60]
Length of array: 5
```

7 WAP To IMPLEMENT STRING MANIPULATION IN PYTHON

7.1 PROGRAM CODE:

```
# String Manipulation in Python
# Creating a string
text = "Hello Python"
print("Original string:", text)
# Access characters
print("First character:", text[0])
# String length
print("Length of string:", len(text))
# Convert to uppercase
print("Uppercase:", text.upper())
# Convert to lowercase
print("Lowercase:", text.lower())
# Replace a word
new_text = text.replace("Python", "World")
print("After replacement:", new_text)
# String concatenation
text2 = " Programming"
combined = text + text2
print("Concatenated string:", combined)
# String slicing
print("Sliced string (0 to 5):", text[0:5])
```

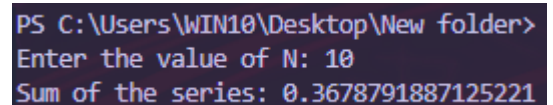


```
PS C:\Users\WIN10\Desktop\New folder> python -
Original string: Hello Python
First character: H
Length of string: 12
Uppercase: HELLO PYTHON
Lowercase: hello python
After replacement: Hello World
Concatenated string: Hello Python Programming
Sliced string (0 to 5): Hello
```

8 WAP To FIND SUM OF THE FOLLOWING SERIES FOR N TERMS: $1 - 2/2! + 3/3! \dots + N/N!$

8.1 PROGRAM CODE:

```
# Program to find sum of the series: 1 - 2/2! + 3/3! - ... + N/N!
import math
n = int(input("Enter the value of N: "))
sum = 0
for i in range(1, n + 1):
    term = i / math.factorial(i)
    if i % 2 == 0:
        sum = sum - term
    else:
        sum = sum + term
print("Sum of the series:", sum)
```



```
PS C:\Users\WIN10\Desktop\New folder>
Enter the value of N: 10
Sum of the series: 0.3678791887125221
```

9 WAP TO CALCULATE THE SUM AND PRODUCT OF TWO COMPATIBLE MATRICES.

9.1 PROGRAM CODE:

```
# Program to calculate sum and product of two matrices

# Input matrix dimensions
r1 = int(input("Enter number of rows of matrices: "))
c1 = int(input("Enter number of columns of matrices: "))

print("Enter elements of Matrix A:")
A = []
for i in range(r1):
    row = []
    for j in range(c1):
        row.append(int(input()))
    A.append(row)

print("Enter elements of Matrix B:")
B = []
for i in range(r1):
    row = []
    for j in range(c1):
        row.append(int(input()))
    B.append(row)

# Matrix Addition
sum_matrix = []
for i in range(r1):
    row = []
    for j in range(c1):
        row.append(A[i][j] + B[i][j])
    sum_matrix.append(row)

# Matrix Multiplication
product_matrix = []
for i in range(r1):
    row = []
    for j in range(c1):
        total = 0
        for k in range(c1):
            total += A[i][k] * B[k][j]
        row.append(total)
    product_matrix.append(row)

print("Sum of Matrices:")
for row in sum_matrix:
    print(row)

print("Product of Matrices:")
for row in product_matrix:
```

```
PS C:\Users\WIN10\Desktop\New folder>
Enter number of rows of matrices: 3
Enter number of columns of matrices: 3
Enter elements of Matrix A:
1
2
3
4
5
6
7
8
9
Enter elements of Matrix B:
1
2
3
4
5
6
7
8
9
Sum of Matrices:
[2, 4, 6]
[8, 10, 12]
[14, 16, 18]
Product of Matrices:
[30, 36, 42]
[66, 81, 96]
[102, 126, 150]
PS C:\Users\WIN10\Desktop\New folder>
```

```
print(row))
```

10 WAP TO CREATE CLASS AND OBJECTS IN PYTHON.

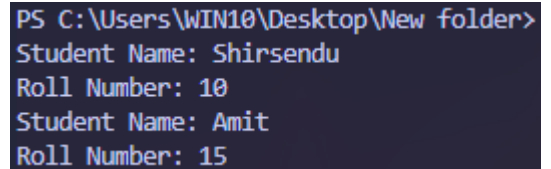
10.1 PROGRAM CODE:

```
# Program to demonstrate Class and Objects in Python
```

```
class Student:
    def __init__(self, name, roll):
        self.name = name
        self.roll = roll
    def display(self):
        print("Student Name:", self.name)
        print("Roll Number:", self.roll)

# Creating objects of the class
s1 = Student("Shirsendu", 10)
s2 = Student("Amit", 15)

# Calling member function using objects
s1.display()
s2.display()
```



```
PS C:\Users\WIN10\Desktop\New folder>
Student Name: Shirsendu
Roll Number: 10
Student Name: Amit
Roll Number: 15
```

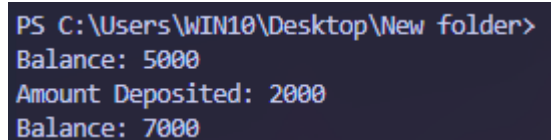
11 WAP TO IMPLEMENT DATA HIDING IN PYTHON.

11.1 PROGRAM CODE:

```
# Program to demonstrate Data Hiding in Python
class Account:
    def __init__(self, balance):
        self.__balance = balance # Private variable
    def show_balance(self):
        print("Balance:", self.__balance)
    def deposit(self, amount):
        self.__balance += amount
        print("Amount Deposited:", amount)

# Creating object of the class
acc = Account(5000)

# Accessing data using member functions
acc.show_balance()
acc.deposit(2000)
acc.show_balance()
```



```
PS C:\Users\WIN10\Desktop\New folder>
Balance: 5000
Amount Deposited: 2000
Balance: 7000
```


12 WAP TO IMPLEMENT CONSTRUCTOR AND DESTRUCTOR FOR A CLASS IN PYTHON.

12.1 PROGRAM CODE:

```
class Student:
    # Constructor
    def __init__(self, name, age):
        self.name = name
        self.age = age
        print(f"Constructor called: Student {self.name} of age {self.age}
created.")
    # Destructor
    def __del__(self):
        print(f"Destructor called: Student {self.name} is destroyed.")
# Creating objects
student1 = Student("Alice", 20)
student2 = Student("Bob", 22)
print(f"{student1.name} is {student1.age} years old.")
print(f"{student2.name} is {student2.age} years old.")
# Deleting objects explicitly
del student1
del student2
print("End of program.")
```

```
PS C:\Users\WIN10\Desktop\New folder> python -u "c:\U
Constructor called: Student Alice of age 20 created.
Constructor called: Student Bob of age 22 created.
Alice is 20 years old.
Bob is 22 years old.
Destructor called: Student Alice is destroyed.
Destructor called: Student Bob is destroyed.
End of program.
```

13 WAP TO IMPLEMENT DIFFERENT TYPES OF INHERITANCE IN PYTHON.

13.1 PROGRAM CODE:

```
# -----
# 1. Single Inheritance
# -----
print("=== Single Inheritance ===")
class Parent:
    def parent_method(self):
        print("This is the parent class method.")
class Child(Parent):
    def child_method(self):
        print("This is the child class method.")
c = Child()
c.parent_method() # inherited from Parent
c.child_method() # own method
# -----
# 2. Multiple Inheritance
# -----
print("\n=== Multiple Inheritance ===")
class Mother:
    def mother_method(self):
        print("This is the mother class method.")
class Father:
    def father_method(self):
```

```

        print("This is the father class method.")
class Child2(Mother, Father):
    def child_method(self):
        print("This is the child class method.")
c2 = Child2()
c2.mother_method()
c2.father_method()
c2.child_method()
# -----
# 3. Multilevel Inheritance
# -----
print("\n=== Multilevel Inheritance ===")
class Grandparent:
    def grandparent_method(self):
        print("This is the grandparent class method.")
class Parent2(Grandparent):
    def parent_method(self):
        print("This is the parent class method.")
class Child3(Parent2):
    def child_method(self):
        print("This is the child class method.")
c3 = Child3()
c3.grandparent_method()
c3.parent_method()
c3.child_method()
# -----
# 4. Hierarchical Inheritance
# -----
print("\n=== Hierarchical Inheritance ===")
class Parent3:
    def parent_method(self):
        print("This is the parent class method.")
class Child4(Parent3):
    def child4_method(self):
        print("This is child 4 class method.")
class Child5(Parent3):
    def child5_method(self):
        print("This is child 5 class method.")
c4 = Child4()
c5 = Child5()
c4.parent_method()
c4.child4_method()
c5.parent_method()
c5.child5_method()
# -----
# 5. Hybrid Inheritance
# -----
print("\n=== Hybrid Inheritance ===")
class A:
    def method_a(self):
        print("Class A method")
class B(A):
    def method_b(self):

```

```

PS C:\Users\WIN10\Desktop\New folder>
=== Single Inheritance ===
This is the parent class method.
This is the child class method.

=== Multiple Inheritance ===
This is the mother class method.
This is the father class method.
This is the child class method.

=== Multilevel Inheritance ===
This is the grandparent class method.
This is the parent class method.
This is the child class method.

=== Hierarchical Inheritance ===
This is the parent class method.
This is child 4 class method.
This is the parent class method.
This is child 5 class method.

=== Hybrid Inheritance ===
Class A method
Class B method
Class C method
Class D method

```

```

        print("Class B method")
class C(A):
    def method_c(self):
        print("Class C method")
class D(B, C):
    def method_d(self):
        print("Class D method")
d = D()
d.method_a() # from A
d.method_b() # from B
d.method_c() # from C
d.method_d() # own method

```

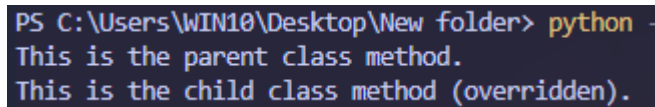
14 WAP TO IMPLEMENT CONCEPT OF OVERRIDING IN PYTHON.

14.1 PROGRAM CODE:

```

# Parent class
class Parent:
    def show(self):
        print("This is the parent class method.")
# Child class
class Child(Parent):
    # Overriding the show() method
    def show(self):
        print("This is the child class method (overridden).")
# Creating objects
p = Parent()
c = Child()
p.show() # Calls parent method
c.show() # Calls child method (overridden)

```



```

PS C:\Users\WIN10\Desktop\New folder> python
This is the parent class method.
This is the child class method (overridden).

```

15 WRITE PROGRAMS TO CREATE MATHEMATICAL 3D OBJECTS USING CLASS.

A. CURVE B. SPHERE C. CONE D. ARROW E. RING F. CYLINDER

15.1 PROGRAM CODE:

```

import math
# A. CURVE (Parametric Helix)
class Curve:
    def __init__(self, radius=1, pitch=0.5):
        self.radius = radius
        self.pitch = pitch # Vertical distance per radian
    def get_point(self, t):
        """Returns the (x, y, z) coordinates for a given parameter t."""
        x = self.radius * math.cos(t)
        y = self.radius * math.sin(t)
        z = self.pitch * t
        return (round(x, 4), round(y, 4), round(z, 4))
# B. SPHERE
class Sphere:

```

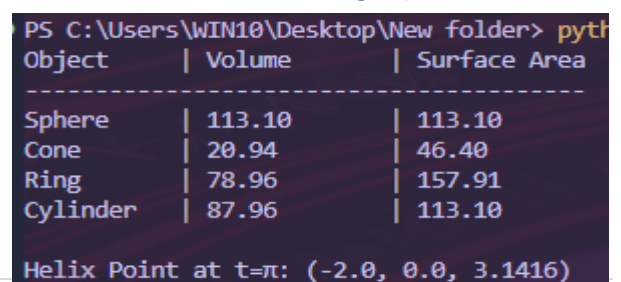
```

def __init__(self, radius=1):
    self.radius = radius

def volume(self):
    return (4/3) * math.pi * pow(self.radius, 3)

def surface_area(self):
    return 4 * math.pi * pow(self.radius, 2)
# C. CONE
class Cone:
    def __init__(self, radius=1, height=2):
        self.radius = radius
        self.height = height
    def volume(self):
        return (1/3) * math.pi * pow(self.radius, 2) * self.height
    def surface_area(self):
        # s is the slant height: s = sqrt(r^2 + h^2)
        slant_height = math.sqrt(self.radius**2 + self.height**2)
        return math.pi * self.radius * (self.radius + slant_height)
# D. ARROW (Compound Object: Cylinder + Cone)
class Arrow:
    def __init__(self, shaft_r=0.1, shaft_h=2, head_r=0.3, head_h=0.5):
        self.shaft = Cylinder(shaft_r, shaft_h)
        self.head = Cone(head_r, head_h)
    def total_volume(self):
        return self.shaft.volume() + self.head.volume()
# E. RING (Torus)
class Ring:
    def __init__(self, major_R=3, minor_r=1):
        self.R = major_R # Distance from center of tube to center of torus
        self.r = minor_r # Radius of the tube itself
    def volume(self):
        # V = (pi * r^2)(2 * pi * R)
        return 2 * pow(math.pi, 2) * self.R * pow(self.r, 2)
    def surface_area(self):
        # A = (2 * pi * r)(2 * pi * R)
        return 4 * pow(math.pi, 2) * self.R * self.r
# F. CYLINDER
class Cylinder:
    def __init__(self, radius=1, height=2):
        self.radius = radius
        self.height = height
    def volume(self):
        return math.pi * pow(self.radius, 2) * self.height
    def surface_area(self):
        return 2 * math.pi * self.radius * (self.radius + self.height)
# --- Output Demonstration ---
if __name__ == "__main__":
    objs = {
        "Sphere": Sphere(3),
        "Cone": Cone(2, 5),
        "Ring": Ring(4, 1),
        "Cylinder": Cylinder(2, 7)

```



Object	Volume	Surface Area
Sphere	113.10	113.10
Cone	20.94	46.40
Ring	78.96	157.91
Cylinder	87.96	113.10

Helix Point at t=π: (-2.0, 0.0, 3.1416)

```

}
print(f"{'Object':<10} | {'Volume':<12} | {'Surface Area':<12}")
print("-" * 40)
for name, obj in objs.items():
    print(f"{'name':<10} | {'obj.volume()':<12.2f} |
{'obj.surface_area()':<12.2f}")
    helix = Curve(radius=2, pitch=1)
    print(f"\nHelix Point at t= $\pi$ : {helix.get_point(math.pi)}")

```

16 WAP TO CHECK IF A NUMBER IS POSITIVE, NEGATIVE OR 0

16.1 PROGRAM CODE:

```

num = float(input("Enter a number: "))
if num > 0:
    print("The number is Positive")
elif num < 0:
    print("The number is Negative")
else:
    print("The number is Zero")

```

```

PS C:\Users\WIN10\Desktop\New folder>
Enter a number: 1
The number is Positive
PS C:\Users\WIN10\Desktop\New folder>
Enter a number: 0
The number is Zero
PS C:\Users\WIN10\Desktop\New folder>
Enter a number: -1
The number is Negative
PS C:\Users\WIN10\Desktop\New folder>

```

17 WAP TO CHECK LEAP YEAR OR NOT

17.1 PROGRAM CODE:

```

year = int(input("Enter a year: "))
if (year % 400 == 0) or (year % 4 == 0 and year % 100 != 0):
    print("It is a Leap Year")
else:
    print("It is NOT a Leap Year")

```

```

PS C:\Users\WIN10\Desktop\New folder>
Enter a year: 2005
It is NOT a Leap Year

```

18 WAP TO DISPLAY CALENDAR OF THE GIVEN MONTH AND YEAR.

18.1 PROGRAM CODE:

```

import calendar
year = int(input("Enter year: "))
month = int(input("Enter month (1-12): "))
print("\nCalendar:\n")
print(calendar.month(year, month))

```

```

PS C:\Users\WIN10\Desktop\New folder>
Enter year: 2005
Enter month (1-12): 03

Calendar:

      March 2005
Mo Tu We Th Fr Sa Su
  1  2  3  4  5  6
  7  8  9 10 11 12 13
 14 15 16 17 18 19 20
 21 22 23 24 25 26 27
 28 29 30 31

```